

Query vs Scan: Quando Usar Cada Um

Otimizando Leituras no DynamoDB

AWS Certified Developer Associate

Instrutor: Marlon Anesi

Query: Busca Eficiente



O que é Query

Busca eficiente usando Partition Key (obrigatório)

Opcionalmente pode usar Sort Key para filtrar

Retorna apenas itens da partição especificada

Características

- Extremamente rápido: acesso direto à partição
- Consome menos RCUs (Read Capacity Units)
- Pode ordenar por Sort Key (ScanIndexForward)
- Suporta filtros na Sort Key (begins_with, between, etc)

Sintaxe básica

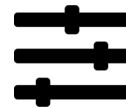
- KeyConditionExpression: obrigatório (PK e opcionalmente SK)
- FilterExpression: opcional (filtros em outros atributos)

Exemplo

Query com PK: "Buscar todos os pedidos do USER#123"

Query com PK+SK: "Buscar pedidos do USER#123 em 2024-01"

Scan: Busca Completa na Tabela



O que é Scan

Lê TODOS os itens da tabela ou índice

Não usa chaves primárias para busca

Examina cada item sequencialmente

Características

- LENTO: examina toda a tabela
- CARO: consome muitos RCUs
- Pode causar throttling em tabelas grandes
- Não é escalável para produção

Quando usar Scan

- Export/backup de toda a tabela
- Análises ad-hoc em ambiente de desenvolvimento
- Tabelas pequenas (< 100 itens)

IMPORTANTE

Evite Scan em produção sempre que possível!

Se precisa fazer Scan frequentemente, redesenhe a tabela

Query vs Scan: Comparação Direta

Aspecto	Query	Scan
Requer PK?	SIM (obrigatório)	NÃO
Performance	RÁPIDA	LENTA
Custo (RCUs)	BAIXO	ALTO
Itens lidos	Apenas 1 partição	Toda a tabela
Escalabilidade	EXCELENTE	RUIM
Ordenação	Por Sort Key	Não ordenado
Uso em produção	SEMPRE	EVITAR

Exemplo de custo

Tabela com 1 milhão de itens:

- Query: retorna 10 itens em 5ms, consome 1 RCU
- Scan: lê 1 milhão de itens em 30s+, consome 100.000 RCUs

Se Precisar Usar Scan: Otimizações

1. Parallel Scan

Divide tabela em segmentos e faz scan paralelo

Reduz tempo total mas ainda consome mesmos RCUs

Especificar TotalSegments e Segment (0 a N-1)

2. FilterExpression

Filtra resultados APÓS ler todos os itens

NÃO reduz RCUs (ainda lê tudo)

Apenas reduz dados retornados para aplicação

3. Limit

Limita número de itens EXAMINADOS (não retornados)

Use paginação com LastEvaluatedKey

4. ProjectionExpression

Retorna apenas atributos específicos

Reduz tráfego de rede mas não RCUs

Melhor solução: REDESENHAR TABELA

Criar índices secundários (GSI/LSI) para queries específicas

Pontos-chave para revisão

1. Query

Usa PK (obrigatório) + SK (opcional)

Rápido, barato, escalável - USE SEMPRE!

2. Scan

Lê toda a tabela, lento, caro

EVITE em produção!

3. Quando usar cada um

Query: 99% dos casos em produção

Scan: exports, análises ad-hoc, dev/test

LEMBRE-SE: Se pode usar Partition Key > sempre use Query! Scan só em último caso!